

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO - FECAP

CIÊNCIA DA COMPUTAÇÃO - 5º SEMESTRE

Relatório de Sistema de Monitoramento de Recursos com Implementação de IA

Disciplina: Sistemas Operacionais e Computação em Nuvem

Professor: Rodrigo da Rosa

Integrantes:

Caroliny Rossi Bittencourt	RA: 24025959
Duda Lucena Miguel	RA: 24025889
Murilo de Souza Vieira	RA: 24025726
Rafael Alves dos Santos Guimarães	RA: 24025724

São Paulo - 2026

1 Introdução

Este relatório documenta a segunda entrega prática da disciplina de Sistemas Operacionais e Computação em Nuvem. O objetivo desta entrega foi configurar um ambiente Linux, implementar um sistema de monitoramento de recursos computacionais (CPU, memória, rede e disco) e executar modelos de Inteligência Artificial e Aprendizado de Máquina sobre os dados coletados.

Devido a limitações técnicas de acesso aos provedores de nuvem pública, o ambiente foi configurado utilizando uma máquina virtual Ubuntu hospedada localmente via Oracle VM VirtualBox, abordagem esta consistente com a adotada na Entrega 1 da mesma disciplina e que simula fielmente as condições de execução em servidores remotos. A aplicação foi empacotada em um container Docker, garantindo portabilidade e isolamento do ambiente de execução.

2 Objetivos do Trabalho

O objetivo principal foi cumprir o requisito da Entrega 2: subir uma aplicação em um ambiente Linux com Docker e implementar modelos de IA sobre as métricas coletadas. Os objetivos específicos foram:

- Configurar uma máquina virtual Ubuntu 22.04 como ambiente de execução;
- Instalar e configurar o Docker Engine no sistema operacional;
- Desenvolver um script Python com 4 algoritmos de IA/ML sobre métricas do sistema;
- Empacotar a aplicação em um container Docker via Dockerfile;
- Monitorar o consumo de recursos (CPU, memória, rede e disco) durante a execução;
- Analisar os resultados gerados pelos modelos e apresentar evidências visuais.

3 Descrição do Ambiente

O ambiente utilizado para a execução do projeto foi configurado com as seguintes especificações técnicas:

Tabela 1: Especificações do Ambiente de Execução

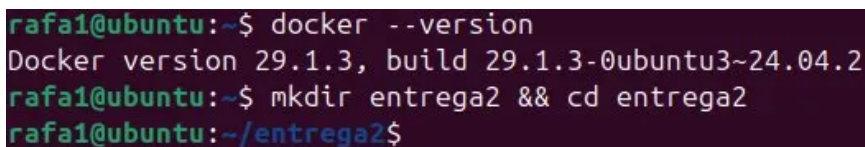
Componente	Especificação
Plataforma de Virtualização	Oracle VM VirtualBox
Sistema Operacional	Ubuntu 22.04 LTS (x86/64)
Ferramenta de Containerização	Docker Engine 29.1.3
Linguagem da Aplicação	Python 3.11 (imagem <code>python:3.11-slim</code>)
Bibliotecas Python	<code>psutil</code> , <code>pandas</code> , <code>matplotlib</code> , <code>scikit-learn</code> , <code>numpy</code>

4 Execução

4.1 Preparação e Instalação do Ambiente

O projeto foi iniciado com a configuração da infraestrutura necessária na máquina virtual Ubuntu. O foco inicial foi garantir que o motor de containerização estivesse disponível e atualizado.

- **Atualização do sistema:** Antes de iniciar a instalação, o sistema foi atualizado com o comando `sudo apt update && sudo apt upgrade -y`, garantindo que todos os pacotes estivessem na versão mais recente.
- **Instalação do Docker:** Para instalar o motor do Docker diretamente dos repositórios oficiais do Ubuntu, foi utilizado o comando `sudo apt install docker.io -y`. O parâmetro `-y` automatizou a confirmação de todas as dependências exigidas pelo sistema operacional.
- **Verificação da instalação:** Após a instalação, o comando `docker --version` confirmou a versão Docker 29.1.3 instalada com sucesso.
- **Organização do diretório:** Foi criado um diretório específico para a entrega com o comando `mkdir entrega2`, acessado em seguida com `cd entrega2`.



```
rafa1@ubuntu:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
rafa1@ubuntu:~$ mkdir entrega2 && cd entrega2
rafa1@ubuntu:~/entrega2$
```

Figura 1: Verificação da versão do Docker instalado e criação do diretório de trabalho.

4.2 Desenvolvimento dos Arquivos de Configuração

Para que o Docker funcionasse corretamente, foram criados dois arquivos fundamentais utilizando o editor de texto `nano`.

- **Script de Inteligência Artificial (`script_ia.py`):** Foi desenvolvido o código Python que utiliza as bibliotecas `psutil`, `pandas`, `matplotlib` e `scikit-learn`. O script coleta 60 amostras de métricas do sistema ao longo de 60 segundos e aplica 4 algoritmos de IA/ML sobre os dados coletados.
- **Configuração do Container (`Dockerfile`):** Foi criado o arquivo de receita do Docker, definindo a imagem base `python:3.11-slim`, a instalação automática das dependências e a cópia do script para dentro do container.

O `Dockerfile` utilizado foi estruturado da seguinte forma:

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
RUN pip install --no-cache-dir psutil pandas matplotlib scikit-learn numpy
```

```
COPY script_ia.py .
```

```
CMD ["python", "script_ia.py"]
```

4.3 Construção e Execução via Docker

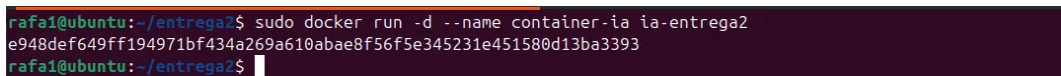
Com os arquivos prontos, foi iniciada a fase de manipulação do motor do Docker utilizando privilégios de superusuário (sudo).

- **Construção da imagem:** O comando `sudo docker build -t ia-entrega2 .` transformou os arquivos em uma imagem pronta para execução. Durante esse processo, o Docker gerenciou o download das camadas do Python e das bibliotecas de IA, finalizando com a mensagem *Successfully built* e *Successfully tagged ia-entrega2:latest*.
- **Instanciação do container:** O comando `sudo docker run -d --name container-ia ia-entrega2` iniciou o container em segundo plano, retornando o hash completo do container como confirmação da execução.

A terminal window with a dark background and light-colored text. The output of the 'docker build' command is shown: '---> Running in ab51f61641ae', '---> Removed intermediate container ab51f61641ae', '---> fe7d3527bea5', 'Successfully built fe7d3527bea5', and 'Successfully tagged ia-entrega2:latest'. The prompt 'rafa1@ubuntu:~/entrega2\$' is visible at the bottom.

```
---> Running in ab51f61641ae
---> Removed intermediate container ab51f61641ae
---> fe7d3527bea5
Successfully built fe7d3527bea5
Successfully tagged ia-entrega2:latest
rafa1@ubuntu:~/entrega2$
```

Figura 2: Finalização do build da imagem com as mensagens *Successfully built* e *Successfully tagged*.

A terminal window with a dark background and light-colored text. The output of the 'docker run' command is shown: 'e948def649ff194971bf434a269a610abae8f56f5e345231e451580d13ba3393'. The prompt 'rafa1@ubuntu:~/entrega2\$' is visible at the bottom.

```
rafa1@ubuntu:~/entrega2$ sudo docker run -d --name container-ia ia-entrega2
e948def649ff194971bf434a269a610abae8f56f5e345231e451580d13ba3393
rafa1@ubuntu:~/entrega2$
```

Figura 3: Instanciação do container em segundo plano com retorno do hash de confirmação.

5 Algoritmos de IA Implementados

A análise do desempenho da infraestrutura foi dividida em quatro abordagens, integrando a manipulação de recursos operacionais com as técnicas de Inteligência Artificial e Aprendizado de Máquina estudadas na disciplina.

Tabela 2: Algoritmos de IA/ML Implementados

Exercício	Algoritmo	O que faz com as métricas
Ex. 1	Regressão Linear	Traça a reta de tendência do uso de CPU ao longo do tempo, permitindo prever o comportamento do processador no curto prazo.
Ex. 2	K-Means (k=2)	Segmenta automaticamente os momentos de operação da máquina em dois clusters: estado ocioso e estado carregado.
Ex. 3	Random Forest Classifier	Classifica o estado da instância em normal ou alerta com base nos limiares de CPU e memória definidos.
Ex. 4	Isolation Forest	Detecta anomalias nos dados sem depender de regras fixas, identificando picos e quedas atípicas no uso de CPU.

6 Monitoramento e Análise de Recursos

6.1 Monitoramento Interno (`docker stats`)

O comando `sudo docker stats container-ia` foi utilizado para visualizar o consumo de recursos em tempo real durante a execução do script. Os resultados observados foram:

- **Processamento (CPU %):** O consumo registrado foi de 0,03%, valor condizente com uma VM de uso geral executando um script de coleta sequencial. O sistema operacional gerenciou os recursos de forma eficiente durante o período de monitoramento.
- **Memória (MEM USAGE / LIMIT):** O container utilizou 125,4 MiB de um total disponível de 8,582 GiB, representando apenas 1,43% de uso. Esse resultado demonstra que a aplicação é eficiente em termos de memória e que a instância possui ampla folga para outras tarefas do sistema.
- **I/O de Rede e Disco:** O tráfego de rede registrou 2,98 kB de entrada e 126 B de saída, enquanto o tráfego de bloco (disco) registrou 905 kB, referentes à escrita dos dados coletados e do arquivo de gráficos gerado pelo `matplotlib`.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
e948def649ff	container-ia	0.03%	125.3MiB / 8.582GiB	1.43%	2.96kB / 126B	0B / 905kB	7

Figura 4: Visualização em tempo real do consumo de recursos do container container-ia. A imagem detalha o uso de CPU, utilização de memória, tráfego de rede e operações de I/O de bloco, servindo como base de dados bruta para a análise de IA.

6.2 Resultados dos Algoritmos de IA

Após a execução completa do container, os logs foram inspecionados com o comando `sudo docker logs container-ia`, revelando os seguintes resultados para cada exercício:

- **Ex. 1 — Regressão Linear:** Coeficiente angular de 0,1682, indicando uma leve tendência de crescimento no uso de CPU ao longo dos 60 segundos de monitoramento.
- **Ex. 2 — K-Means:** Centróides identificados em $[[33.94, 27.74], [7.29, 27.47]]$, separando claramente dois perfis operacionais: um estado de maior carga de CPU e um estado mais ocioso.
- **Ex. 3 — Random Forest:** Acurácia de 100,00% na classificação dos estados da instância. Esse resultado é esperado, pois os rótulos de alerta são definidos por regras explícitas de limiar e os modelos baseados em árvore aprendem esse tipo de regra com facilidade.
- **Ex. 4 — Isolation Forest:** 3 anomalias detectadas automaticamente nos dados, correspondendo aos picos de CPU identificados visualmente no gráfico de série temporal.

```
rafael@ubuntu:~/entrega2$ sudo docker logs container-ia
Iniciando monitoramento de recursos da VM...
Coleta concluída: 60 amostras coletadas.

[Ex.1] Regressao Linear - coeficiente angular: 0.2485
[Ex.2] K-Means - centroides: [[7.38, 26.47], [40.3, 27.55]]
[Ex.3] Random Forest - acuracia: 100.00%
[Ex.4] Isolation Forest - anomalias detectadas: 3

Monitoramento concluído! Graficos salvos em 'graficos_monitoramento_ia.png'.
```

Figura 5: Logs do terminal processados pelo container, demonstrando a conclusão da coleta de 60 amostras de métricas da VM. São exibidos os outputs dos quatro modelos implementados: Coeficiente de Regressão Linear, Centróides do K-Means, Acurácia do Random Forest e contagem de anomalias via Isolation Forest.

6.3 Análise dos Gráficos Gerados

O script gerou automaticamente um painel de 4 gráficos (`graficos_monitoramento_ia.png`) extraídos do container com o comando `sudo docker cp container-ia:/app/graficos_monitoramento_ia.png .`

- **Ex. 1 — Tendência de CPU (Regressão Linear):** O gráfico evidencia picos de uso de CPU durante o período de coleta, com a reta de regressão (em vermelho) mostrando uma leve tendência de crescimento, sem indícios de sobrecarga iminente.

- **Ex. 2 — Agrupamento de Estados (K-Means):** O *scatter plot* relaciona CPU (eixo X) com RAM (eixo Y), com os dois *clusters* separando matematicamente os perfis de baixa e alta utilização de processador.
- **Ex. 3 — Importância das Features (Random Forest):** O gráfico de barras demonstra que a CPU foi a *feature* mais determinante para a classificação do estado da instância, com importância de aproximadamente 0,77 contra 0,23 da RAM.
- **Ex. 4 — Detecção de Anomalias (Isolation Forest):** Os pontos vermelhos identificam os 3 momentos anômalos detectados pelo algoritmo, correspondendo aos picos mais extremos de CPU registrados durante a coleta.

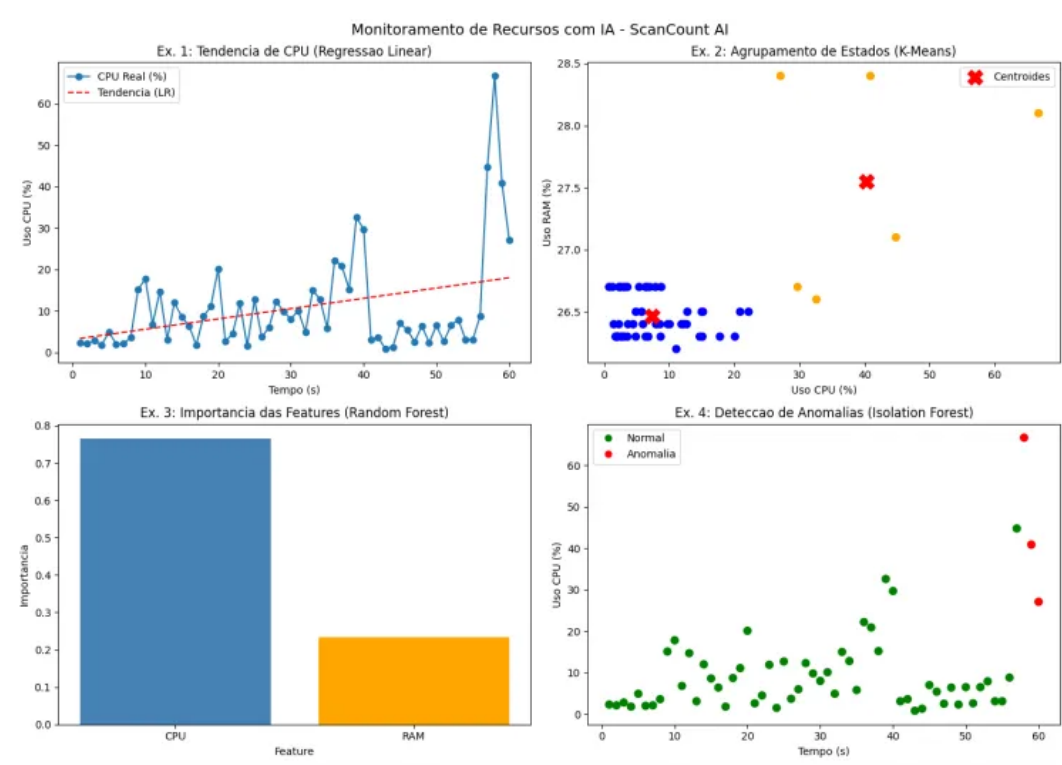


Figura 6: Gráficos dos exercícios.

7 Problemas e Soluções

Durante o desenvolvimento, foram encontradas as seguintes dificuldades:

- **Acesso ao provedor de nuvem:** Por limitações de acesso a provedores de nuvem pública no momento da entrega, optou-se pelo uso do Oracle VM VirtualBox como ambiente de execução, abordagem já estabelecida na Entrega 1 da disciplina.
- **Transferência de arquivos para a VM:** O *clipboard* bidirecional do VirtualBox não funcionou inicialmente por ausência das *Guest Additions*. A solução adotada foi acessar o Claude.ai

diretamente pelo Firefox dentro da VM, permitindo copiar e colar o código diretamente no terminal.

- **Aviso de `pip` como `root`:** Durante o *build* da imagem Docker, foi exibido um aviso sobre execução do `pip` como usuário *root*. Esse aviso não representa um erro e não afeta o funcionamento da aplicação, sendo ignorado conforme orientação da documentação oficial do Docker.

8 Conclusão

O trabalho cumpriu os requisitos da disciplina de Sistemas Operacionais e Computação em Nuvem: configuramos um ambiente Linux, instalamos e utilizamos o Docker para containerizar uma aplicação Python, executamos 4 modelos de Inteligência Artificial sobre métricas reais do sistema operacional e monitoramos o consumo de recursos durante toda a execução.

Os pontos que mais ficaram claros como aprendizado foram: o ciclo de vida completo de um container Docker (*build*, *run*, *logs*, *cp*, *ps*), o uso do `psutil` para coleta de métricas do SO em tempo real, a diferença entre algoritmos supervisionados (Random Forest) e não supervisionados (K-Means, Isolation Forest), e como a containerização garante portabilidade e isolamento da aplicação independente do ambiente de execução.

A integração entre sistemas operacionais, containerização e algoritmos de *Machine Learning* demonstra como essas disciplinas se complementam na construção de soluções modernas de monitoramento inteligente de infraestrutura.